

# BACKEND APPLICATION DEVELOPMENT WITH JAVA SPRING BOOT

---

*A Comprehensive Instructional Guide*

Spring Data JPA | MySQL Driver | Lombok | Spring Web

**By Mr. Jefferson Youashi**

*OSS Developer*

## Introduction

Welcome to this course on Backend Application Development with Java Spring Boot. In this module, we are going to lay a solid theoretical foundation before we dive into any hands-on coding. I want you to read through these notes carefully, understand every concept, and be able to explain them back to me — because when we sit down to write code, I expect you to already know *why* we are writing what we write.

Backend development is the backbone of any software application. It is the part of the system that the end user never sees, but it is responsible for everything that happens under the hood — storing data, processing business logic, exposing endpoints for communication, and ensuring that the application behaves correctly and securely.

We will be working with the Spring Boot framework, which is a powerful, production-grade framework built on top of the Spring ecosystem. Rather than managing raw database connections with JDBC, we will be leveraging four important dependencies that will handle much of the complexity for us. These are:

- **Spring Data JPA** — for persisting data in relational databases using the Java Persistence API.
- **MySQL Driver** — which provides the JDBC connector that allows our application to communicate with a MySQL database.
- **Lombok** — a developer tool that reduces boilerplate code through Java annotations.
- **Spring Web** — which allows us to build RESTful web services using Spring MVC, running on an embedded Apache Tomcat server.

---

## Chapter 1: Understanding the Spring Boot Architecture

### 1.1 What is Spring Boot?

Spring Boot is an opinionated framework built on top of the Spring Framework. What we mean by 'opinionated' is that it makes many configuration decisions for you, so that you can focus on writing business logic rather than spending time on infrastructure setup.

When you create a Spring Boot application, it comes with an embedded web server — Apache Tomcat by default — meaning you do not need to install and configure a separate server. You simply run your application as a regular Java program, and the server starts up automatically.

Spring Boot follows a layered architecture pattern. This is a design approach where the application is divided into distinct layers, each with a specific responsibility. The three core layers we will work with are:

1. **The Controller Layer** — This is the layer that receives HTTP requests from the outside world. It is the entry point of your API.
2. **The Service Layer** — This is where your business logic lives. The controller delegates work to the service, and the service processes the logic.

3. **The Repository Layer** — This is the data access layer. It is responsible for all communication with the database.

## 1.2 What is the Role of Spring Data JPA?

JPA stands for Java Persistence API. It is a specification — essentially a set of rules and interfaces — that defines how Java objects should be mapped to relational database tables. Hibernate is the most popular implementation of JPA, and Spring Data JPA wraps Hibernate with even more convenience.

When we say we are using Spring Data JPA instead of raw JDBC, we mean that instead of writing raw SQL queries like `SELECT * FROM users WHERE id = 1`, we will describe our database tables as Java classes, and Spring will handle the SQL generation automatically.

This approach is called Object Relational Mapping, or ORM. It maps Java objects to database tables. An instance of a Java class represents a row in a table. Each field of that class corresponds to a column in that table.

---

## Chapter 2: Defining Your Data Model — The Entity Layer

### 2.1 The Concept of an Entity

In the context of JPA, an entity is simply a Java class that is mapped to a database table. When you declare a class as an entity, you are telling Spring: 'This class represents a table in my database. Every object of this class is a row in that table.'

Think of it this way. If you have a `users` table in your MySQL database with columns for `id`, `name`, and `email`, you would create a Java class called `User` with those same fields — and then annotate it so Spring knows the relationship.

### 2.2 Key Entity Annotations

**@Entity**

Declares a Java class as a JPA entity, meaning it will be mapped to a database table.

The `@Entity` annotation is placed at the top of a Java class. It comes from the `javax.persistence` (or `jakarta.persistence` in newer versions) package. When Spring Boot starts, it scans for all classes annotated with `@Entity` and registers them as managed entities. This means Hibernate will know how to translate them to SQL.

Every class annotated with `@Entity` must have a primary key. If you do not define one, your application will fail to start.

**@Table**

Specifies the name of the database table that this entity maps to. If omitted, JPA uses the class name by default.

You use `@Table(name = "users")` when the name of your Java class does not match the name of your database table, or when you want to be explicit and clear about the mapping. This is a good practice for maintainability.

For example, you might have a class named `AppUser` in Java, but the corresponding table in the database is called `users`. The `@Table` annotation bridges that gap.

`@Id`

Marks the field in the entity class that corresponds to the primary key column of the database table.

Every entity must have exactly one field marked with `@Id`. This tells JPA which field serves as the unique identifier for each record. Without this, JPA cannot distinguish between rows and your application will throw an error at startup.

`@GeneratedValue`

Specifies the strategy for automatically generating the primary key value. The most common strategy is `GenerationType.IDENTITY`, which delegates key generation to the database (e.g., `AUTO_INCREMENT` in MySQL).

When you annotate your primary key with `@GeneratedValue(strategy = GenerationType.IDENTITY)`, you are saying: 'I do not want to manually assign IDs. Let the database handle this automatically.' In MySQL, this corresponds to the `AUTO_INCREMENT` column property, where each new row gets an automatically incremented integer as its ID.

`@Column`

Maps a specific Java field to a specific column in the database table. Allows customization of column name, constraints, and behaviour.

With `@Column`, you can control several things. You can specify the exact column name using `name`, enforce that a value is required using `nullable = false`, restrict the length of a string using `length`, and enforce that values must be unique using `unique = true`. For example:

`@Column(name = "email_address", nullable = false, unique = true, length = 100)`.

## 2.3 A Complete Entity Example

Let us now look at how all of these annotations come together in a real Java class. Study this carefully:

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "full_name", nullable = false, length = 100)
```

```

private String fullName;

@Column(name = "email", nullable = false, unique = true)
private String email;

@Column(name = "password", nullable = false)
private String password;

// Getters, Setters, Constructors...
}

```

When Hibernate reads this class, it knows the following: the table name is users. The id column is the primary key and auto-increments. The full\_name column cannot be null and has a maximum of 100 characters. The email column must be unique across all rows.

---

## Chapter 3: Reducing Boilerplate with Lombok

### 3.1 The Problem Lombok Solves

In Java, whenever you create a class with private fields, you are expected to write a constructor that sets those fields, getter methods that return those field values, setter methods that update those field values, and sometimes a toString() method for readable output. For a class with five fields, this means you might write thirty or more lines of repetitive, formulaic code that carries no real business value.

Lombok is a Java library that uses annotations to generate this boilerplate code at compile time. You write the annotation, and Lombok generates the getters, setters, constructors, and more — automatically — without cluttering your source file.

### 3.2 Key Lombok Annotations

#### @Data

A convenience annotation that bundles @Getter, @Setter, @ToString, @EqualsAndHashCode, and @RequiredArgsConstructor into a single annotation.

This is the most commonly used Lombok annotation. When you place @Data on a class, Lombok generates all the getter and setter methods for every field, a toString() method that prints the field values, equals() and hashCode() methods based on all fields, and a constructor for all fields marked as final.

Note: Be careful when using @Data on entity classes. The generated equals() and hashCode() methods can sometimes cause unexpected behaviour with JPA proxies. Many developers prefer to use @Getter and @Setter separately on entity classes.

#### @Getter / @Setter

Generates getter and setter methods for all fields. Can also be applied to individual fields.

|                            |                                                                                                                                 |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>@NoArgsConstructor</b>  | Generates a constructor with no arguments. Required by JPA — Hibernate needs a no-argument constructor to instantiate entities. |
| <b>@AllArgsConstructor</b> | Generates a constructor that accepts all fields as arguments. Useful for creating fully populated objects.                      |
| <b>@Builder</b>            | Generates a builder pattern for the class, allowing fluent, readable object construction.                                       |

With **@Builder**, instead of calling a constructor with many arguments in a potentially confusing order, you can build an object like this:

```
User user = User.builder()
    .fullName("Jefferson Youashi")
    .email("jefferson@example.com")
    .password("securepassword")
    .build();
```

This is far more readable, especially when a class has many fields.

---

## Chapter 4: The Repository Layer — Data Access

### 4.1 What is a Repository?

The repository layer is responsible for all communication with the database. It is the only layer that is allowed to talk directly to the database. This separation of concerns is a fundamental principle of clean software architecture — your service layer should not know or care about how data is stored; it simply asks the repository for data and the repository handles the rest.

In Spring Data JPA, we create repository interfaces rather than concrete classes. We define an interface and extend one of Spring's built-in repository interfaces — most commonly `JpaRepository`. Spring then generates the implementation automatically at runtime. You do not have to write a single SQL query for standard operations.

### 4.2 The @Repository Annotation

|                    |                                                                                                                                                                                 |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>@Repository</b> | Marks a class or interface as a Spring-managed repository component — a Data Access Object (DAO). It also enables Spring's exception translation mechanism for database errors. |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

When you annotate an interface with **@Repository**, Spring registers it as a bean in its application context — meaning Spring manages its lifecycle and makes it available for injection wherever it is needed. It also enables exception translation, which converts database-specific exceptions into Spring's unified `DataAccessException` hierarchy.

## 4.3 JpaRepository — Your Ready-Made Toolkit

When your repository interface extends `JpaRepository`, you immediately inherit a rich set of database operations without writing any code. These include:

- `save(entity)` — Inserts a new record or updates an existing one.
- `findById(id)` — Retrieves a record by its primary key.
- `findAll()` — Retrieves all records from the table.
- `deleteById(id)` — Deletes the record with the given ID.
- `existsById(id)` — Returns true if a record with that ID exists.
- `count()` — Returns the total number of records in the table.

You can also define custom query methods simply by following Spring Data's method naming convention. For example:

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    // Spring generates the SQL automatically from the method name
    Optional<User> findByEmail(String email);

    List<User> findByFullNameContaining(String keyword);

    boolean existsByEmail(String email);
}
```

Spring reads the method name and generates the equivalent SQL query. `findByEmail` translates to `SELECT * FROM users WHERE email = ?`. This is the power of Spring Data JPA.

---

## Chapter 5: The Service Layer — Business Logic

### 5.1 What is a Service?

The service layer sits between the controller and the repository. It is the brain of the application. Every piece of business logic — validation, calculations, conditional processing, orchestrating multiple repository calls — belongs in the service layer.

Think of it this way: the controller's job is to receive a request and hand it off. The repository's job is to talk to the database. The service's job is to do everything in between. This clean separation makes your code easier to test, easier to maintain, and easier to extend.

### 5.2 The `@Service` Annotation

`@Service`

Marks a class as a service component in the Spring application context. It is a specialization of the `@Component` annotation, indicating that the class holds business logic.

Functionally, `@Service` behaves like `@Component` — it registers the class as a Spring bean. The distinction is semantic: it communicates intent. When another developer reads your code and sees `@Service`, they immediately understand that this class contains business logic. This improves code readability and maintainability.

### 5.3 Dependency Injection with `@Autowired`

#### `@Autowired`

Instructs Spring to automatically inject a dependency — a collaborating bean — into the annotated field, constructor, or setter method.

This is one of the most important annotations in the entire Spring framework. To understand it, you first need to understand what Dependency Injection (DI) is.

Dependency Injection is a design pattern where an object does not create its own dependencies. Instead, those dependencies are provided to it from the outside. Spring manages all the objects (beans) in your application and wires them together automatically.

For example, your `UserService` needs a `UserRepository` to access the database. Without dependency injection, you would write `new UserRepository()` inside your service — which is bad practice because it creates tight coupling. With `@Autowired`, you simply declare that you need a `UserRepository`, and Spring finds the appropriate bean and injects it for you.

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public User createUser(User user) {
        if (userRepository.existsByEmail(user.getEmail())) {
            throw new RuntimeException("Email already in use.");
        }
        return userRepository.save(user);
    }

    public List<User> getAllUsers() {
        return userRepository.findAll();
    }
}
```

Note how the service uses the repository to check for duplicates before saving. That is business logic — and it belongs here, not in the controller and not in the repository.

Modern best practice favors constructor injection over field injection with `@Autowired`. With Lombok's `@RequiredArgsConstructor`, declaring a field as final and letting Lombok generate the constructor achieves the same result in a cleaner way. However, field injection with `@Autowired` is still widely used and perfectly valid for learning purposes.

---

## Chapter 6: The Controller Layer — Exposing REST Endpoints



## 6.1 What is a REST API?

REST stands for Representational State Transfer. A REST API is a way of designing web services so that clients — mobile apps, frontend web applications, other servers — can communicate with your backend using standard HTTP methods.

The four fundamental HTTP methods you will use are:

- **GET** — Retrieve data from the server. (Read operation)
- **POST** — Send data to the server to create a new resource. (Create operation)
- **PUT / PATCH** — Send data to the server to update an existing resource. (Update operation)
- **DELETE** — Request the server to delete a resource. (Delete operation)

## 6.2 Key Controller Annotations

### `@RestController`

A combination of `@Controller` and `@ResponseBody`. Marks the class as a controller where every method returns data directly in the HTTP response body, serialized as JSON by default.

When you annotate a class with `@RestController`, you are telling Spring two things. First, this class contains methods that handle HTTP requests — it is a controller. Second, the return value of every method in this class should be written directly into the HTTP response body as JSON, not rendered as a view or HTML template. This is the standard annotation for building REST APIs in Spring Boot.

### `@RequestMapping`

Maps HTTP requests to specific handler classes or methods. When applied at the class level, it sets a base path for all endpoints within that controller.

You will typically see `@RequestMapping("/api/users")` at the top of a controller class. This means every endpoint in that class will be prefixed with `/api/users`. So if a method inside handles GET requests, the full URL would be `/api/users`. If another method inside handles `/profile`, the full URL would be `/api/users/profile`.

### `@GetMapping()`

A shortcut annotation for `@RequestMapping(method = RequestMethod.GET)`. Maps HTTP GET requests to the annotated method.

You use `@GetMapping("/all")` to define an endpoint that responds to GET requests at the specified path. Since GET is used for retrieving data, a method annotated with this should return data — a list, a single object, or some other response. You can also use `@GetMapping` without a path argument, in which case it maps to the base path of the controller.

Similarly, you should know the following companion annotations:

|                               |                                                                    |
|-------------------------------|--------------------------------------------------------------------|
| <code>@PostMapping()</code>   | Maps HTTP POST requests. Used to create new resources.             |
| <code>@PutMapping()</code>    | Maps HTTP PUT requests. Used to fully update an existing resource. |
| <code>@DeleteMapping()</code> | Maps HTTP DELETE requests. Used to delete a resource.              |
| <code>@PatchMapping()</code>  | Maps HTTP PATCH requests. Used for partial updates to a resource.  |

### 6.3 Additional Controller Annotations

|                            |                                                                                                                                                                                     |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>@PathVariable</code> | Extracts a value from the URL path and binds it to a method parameter. For example, <code>/users/{id}</code> — the <code>{id}</code> segment is extracted and passed to the method. |
| <code>@RequestBody</code>  | Extracts the body of the HTTP request and automatically deserializes it from JSON into a Java object. Used with POST and PUT endpoints.                                             |
| <code>@RequestParam</code> | Extracts query parameters from the URL. For example, <code>/users?page=1</code> — the value 1 is extracted from the page parameter.                                                 |

### 6.4 A Complete Controller Example

```

@RestController
@RequestMapping("/api/users")
public class UserController {

    @Autowired
    private UserService userService;

    // GET /api/users - returns all users
    @GetMapping
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }

    // GET /api/users/5 - returns user with ID 5
    @GetMapping("/{id}")
    public User getUserById(@PathVariable Long id) {
        return userService.findById(id);
    }

    // POST /api/users - creates a new user
    @PostMapping

```

```
public User createUser(@RequestBody User user) {
    return userService.createUser(user);
}

// DELETE /api/users/5 - deletes user with ID 5
@DeleteMapping("/{id}")
public void deleteUser(@PathVariable Long id) {
    userService.deleteById(id);
}
}
```

Notice how clean and readable this controller is. Each method does exactly one thing. The controller does not contain any business logic or database code — it simply delegates to the service.

---

## Chapter 7: The Complete Flow — From Request to Database

### 7.1 Tracing a Request Through the Layers

Let us trace what happens when a client sends a POST request to `/api/users` to create a new user. This will tie together everything we have learned.

4. The client sends an HTTP POST request to `/api/users`, with a JSON body containing name, email, and password.
5. Spring Boot receives the request. It checks its registered controllers and finds `UserController`, whose base path is `/api/users`.
6. Inside `UserController`, it finds the `@PostMapping` method. The `@RequestBody` annotation tells Spring to deserialize the JSON body into a `User` object.
7. The controller calls `userService.createUser(user)`. Control passes to the service layer.
8. The service validates the data — checks if the email already exists, applies any business rules.
9. The service calls `userRepository.save(user)`. Control passes to the repository layer.
10. Spring Data JPA translates the `save()` call into a SQL INSERT statement and executes it against the MySQL database via the MySQL JDBC Driver.
11. The saved `User` object (now with a generated ID) is returned back up through the layers: repository → service → controller.
12. The controller returns the `User` object. Spring serializes it to JSON and writes it to the HTTP response.
13. The client receives a 200 OK response with the newly created user as JSON.

This is the complete request lifecycle in a Spring Boot REST API. Every layer has a clear, single responsibility. The controller handles HTTP. The service handles logic. The repository handles data.

---

## Chapter 8: Annotations Quick Reference Summary

Before the practical session, let us do a complete review. You should be able to state the purpose of each of these annotations without looking at your notes.

|                                |                                                                                                        |
|--------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>@Entity</code>           | Declares a class as a JPA entity mapped to a database table.                                           |
| <code>@Table (name)</code>     | Specifies the database table name for an entity.                                                       |
| <code>@Id</code>               | Marks the primary key field of an entity.                                                              |
| <code>@GeneratedValue</code>   | Configures automatic primary key generation (e.g., AUTO_INCREMENT).                                    |
| <code>@Column</code>           | Maps a field to a specific database column with optional constraints.                                  |
| <code>@Repository</code>       | Marks a class/interface as a data access component (DAO). Enables exception translation.               |
| <code>@Service</code>          | Marks a class as a service component containing business logic.                                        |
| <code>@RestController</code>   | Marks a class as a REST controller; combines <code>@Controller</code> and <code>@ResponseBody</code> . |
| <code>@RequestMapping</code>   | Maps HTTP requests to a controller or method; sets the base URL path.                                  |
| <code>@GetMapping ()</code>    | Maps HTTP GET requests to a method.                                                                    |
| <code>@PostMapping ()</code>   | Maps HTTP POST requests to a method.                                                                   |
| <code>@PutMapping ()</code>    | Maps HTTP PUT requests to a method.                                                                    |
| <code>@DeleteMapping ()</code> | Maps HTTP DELETE requests to a method.                                                                 |
| <code>@Autowired</code>        | Injects a Spring-managed bean automatically into a field, constructor, or setter.                      |
| <code>@PathVariable</code>     | Binds a URI path segment to a method parameter (e.g., /users/{id}).                                    |

|                                           |                                                                                                                    |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>@RequestBody</code>                 | Deserializes the HTTP request body from JSON into a Java object.                                                   |
| <code>@RequestParam</code>                | Extracts a query parameter from the URL (e.g., <code>?page=1</code> ).                                             |
| <code>@Data</code> (Lombok)               | Generates getters, setters, <code>toString</code> , <code>equals</code> , <code>hashCode</code> , and constructor. |
| <code>@NoArgsConstructor</code> (Lombok)  | Generates a no-argument constructor. Required by JPA.                                                              |
| <code>@AllArgsConstructor</code> (Lombok) | Generates a constructor with all fields as arguments.                                                              |
| <code>@Builder</code> (Lombok)            | Generates a builder pattern for fluent, readable object construction.                                              |

---

## Closing Notes

You should understand that every annotation we have covered in these notes exists for a reason. None of them are arbitrary. They are the result of years of software engineering experience, distilled into a framework that solves real, recurring problems in backend development.

When we move to the practical session, you will see these annotations in action. You will create your first entity, wire up your first repository, write your first service, and expose your first REST endpoint. If you have understood these notes, that session will be a confirmation of what you already know — not a surprise.

Read these notes again before we meet for the hands-on session. Be curious. Ask questions. And remember: a developer who understands the 'why' behind the tools they use will always build better software than one who simply copies code.

— Mr. Jefferson Youashi | OSS Developer